LangChain: Runnables & LCEL

LangChain Expression Language for Composable AI Systems

Section 03 `langchain-core v1.2.x`

March 2026

Building composable, type-safe AI applications with LCEL

Agenda

- **What is LCEL?** The mental model
- **The Runnable protocol** — invoke, batch, stream, async variants
- **The pipe operator** — — composition syntax
- **Building chains** — prompt — model — parser
- **Data flow patterns** — Passthrough, Parallel, Lambda, Branch
- **Resilience** — fallbacks, retries, configuration
- **Advanced patterns** — complex chains, debugging, performance
- **Comparison** — LCEL vs legacy Chain classes
- **Cheat sheet & self-check**

Coverage: LangChain v1.2.x (Core + Community packages)

What is LCEL?

LCEL = LangChain Expression Language

Definition

A declarative API for composing language model applications into modular, reusable, and type-safe data pipelines using a consistent **Runnable protocol**.

Three pillars:

1. **Runnable Protocol** — standardized interface for any component
2. **Pipe Operator** — — chainable, readable syntax
3. **Type Safety** — input/output schemas ensure compatibility

Think: Unix pipes, but for LLM workflows.

Why LCEL? The Problem

Legacy Chain classes had pain points:

- Different ‘.run()‘ / ‘.call()‘ signatures across chain types
- No standard way to compose chains together
- Async support was inconsistent (or missing)
- Streaming didn’t work reliably
- Type hints were weak; runtime errors common
- Hard to test, debug, and reuse components

LCEL solves this:

- One protocol, one syntax, unlimited flexibility
- Compose anything into anything (prompt + model + parser)
- First-class async and streaming
- Input/output schemas define contracts
- Easy to test, debug, and share

The Unix Philosophy Applied to LLMs

```
$ echo "What is gravity?" | curl-llm | json-parser
```

Unix pipe	**LCEL equivalent**
`echo` (text source)	Prompt, input dict
`|` (pipe)	`|` (same syntax!)
`curl-llm` (process)	LLM, custom Runnable
`json-parser` (transform)	Parser, RunnableLambda

The mental model: data flows left-to-right; each stage transforms and passes to the next.

Runnable: The Universal Interface

Core Methods

```
Asynchronous methods async .ainvoke(input) -> output async .abatch(inputs: List) ->
List[output] async .astream(input) -> AsyncIterator[output]
```

Key idea: Every LangChain component implements this protocol.

- LLMs, prompts, parsers, retrievers, tools, custom functions
- All composable; all have the same interface
- Mix sync/async freely (LCEL handles the plumbing)

Runnable Protocol: Sync Async

Example: Prompt + LLM

```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

prompt = ChatPromptTemplate.from_template(
    "Explain {topic} in 2 sentences."
)
model = ChatOpenAI(model="gpt-4")
chain = prompt | model

# Sync: invoke returns one output
output = chain.invoke({"topic": "gravity"})

# Batch: invoke multiple inputs
outputs = chain.batch(
    [{"topic": "gravity"}, {"topic": "light"}]
)

# Async: handle thousands concurrently
output = await chain.ainvoke({"topic": "gravity"})
```

Streaming: Real-time Token Output

Streaming with .stream()

```python
# Synchronous streaming (print tokens as they arrive)
for chunk in chain.stream({"topic": "quantum physics"}):
    print(chunk.content, end="", flush=True)

# Async streaming (for web apps, async servers)
async for chunk in chain.astream({"topic": "quantum physics"}):
    print(chunk.content, end="", flush=True)

# Batch + stream: process multiple inputs with progress
for output in chain.batch([...], {max_concurrency: 5}):
    # Each output is fully streamed before next starts
    print(output)
```

Streaming is critical for user-facing LLM apps (show tokens as they're generated, not all at once).

When to Use Each Method

Method	Use case	Example
`invoke()`	Single input, wait for response	API endpoint
`batch()`	Many inputs, maximize throughput	Processing dataset
`stream()`	Single input, show tokens live	Chat UI, notebooks
`ainvoke()`	Async web app, non-blocking	FastAPI endpoint
`abatch()`	Async, many inputs	Async worker pool
`astream()`	Async, live tokens	WebSocket chat

Rule of thumb: Use `batch()` for throughput, `stream()` for UX, `async` for scalability.

The Pipe Operator: Syntax & Semantics

Pipe Composability

```
# Pipe operator: a | b creates a RunnableSequence
# Data flows: a.invoke(input) -> b.invoke(result)

chain = prompt | model | output_parser

# Equivalent to:
# Step 1: prompt.invoke({"topic": "..."})
#   -> PromptValue
# Step 2: model.invoke(PromptValue)
#   -> AIMessage(content="...")
# Step 3: output_parser.invoke(AIMessage)
#   -> Parsed output (str, dict, object)
```

Key Insight

The pipe operator is **syntactic sugar**. Under the hood, it creates a `RunnableSequence` that orchestrates the flow.

Pipe Operator: How It Works

Under the Hood

```python
from langchain_core.runnables import RunnableSequence

# These are equivalent:
chain1 = prompt | model | parser

chain2 = RunnableSequence(
    first=RunnableSequence(
        first=prompt,
        last=model
    ),
    last=parser
)

# Both have .invoke(), .stream(), .batch(), etc.
output1 = chain1.invoke({"topic": "gravity"})
output2 = chain2.invoke({"topic": "gravity"})
# output1 == output2
```

The pipe operator nests automatically; LCEL handles intermediate steps.

Pipe: More Than Sequence

Pipe with Operators

```
# Pipe + dict shorthand creates RunnableParallel
parallel_chain = {
    "question": RunnablePassthrough(),
    "context": retriever
} | prompt | model

# Pipe + conditional creates RunnableBranch
branch_chain = (
    condition_runnable
    | {
        "path_a": chain_a,
        "path_b": chain_b
    }
)

# Pipe + add creates RunnableSequence with assignment
chain = prompt | model | {"original": RunnablePassthrough(), "parsed":
    parser}
```

The Classic Pattern: Prompt — Model — Parser

Complete Example

```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

# 1. Define prompt template
prompt = ChatPromptTemplate.from_template(
    "Explain {topic} in 2 sentences."
)

# 2. Choose LLM
model = ChatOpenAI(model="gpt-4", temperature=0.7)

# 3. Add parser
parser = StrOutputParser()

# 4. Compose
chain = prompt | model | parser

# 5. Run
result = chain.invoke({"topic": "machine learning"})
print(result)  # "Machine learning is..."
```

Input/Output Schemas

Schema Inspection

```python
# Every Runnable has input_schema and output_schema
chain = prompt | model | parser

print(chain.input_schema.schema())
# {
#   "type": "object",
#   "properties": {
#     "topic": {"type": "string"}
#   }
# }

print(chain.output_schema.schema())
# {"type": "string"}

# Schemas enable:
# - Type checking before runtime
# - Auto-documentation
# - Request validation in APIs
# - IDE autocompletion
```

Real-world: RAG Chain

Retrieval-Augmented Generation

```python
from langchain.vectorstores import Chroma
from langchain_core.runnables import RunnablePassthrough

# 1. Retriever
retriever = Chroma.from_documents(...).as_retriever()

# 2. Prompt with context placeholder
prompt = ChatPromptTemplate.from_template("""
    Answer based on context:
    Context: {context}
    Question: {question}
""")

# 3. Format: pass through Q, retrieve context
chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt
    | model
    | StrOutputParser()
)
```

RunnablePassthrough: Identity Transformation

Passing Data Through

```python
from langchain_core.runnables import RunnablePassthrough

# Identity: returns input unchanged
passthrough = RunnablePassthrough()
output = passthrough.invoke("hello")
# output == "hello"

# In a chain: used with dict to keep input available
chain = (
    {"input": RunnablePassthrough(), "output": model}
    | format_results_template
)

# With .assign(): add computed fields without losing input
chain = (
    {"input": RunnablePassthrough()}
    | RunnablePassthrough.assign(
        computed=lambda x: len(x["input"].split())
    )
)
# Returns: {"input": "...", "computed": 5}
```

RunnableParallel: Multiple Branches

Running Tasks in Parallel

```python
from langchain_core.runnables import RunnableParallel

# Dict shorthand automatically creates RunnableParallel
parallel = {
    "question": RunnablePassthrough(),
    "length": RunnableLambda(lambda x: len(x)),
    "upper": RunnableLambda(lambda x: x.upper())
}

result = parallel.invoke("hello")
# {
#   "question": "hello",
#   "length": 5,
#   "upper": "HELLO"
# }

# Explicitly: RunnableParallel(...)
explicit = RunnableParallel(
    question=RunnablePassthrough(),
    length=RunnableLambda(len)
)
```

RunnableLambda: Wrapping Python Functions

Converting Functions to Runnables

```python
from langchain_core.runnables import RunnableLambda

# Wrap a function as a Runnable
def uppercase(x):
    return x.upper()

runnable = RunnableLambda(uppercase)
output = runnable.invoke("hello")
# "HELLO"

# Async functions too
async def fetch_data(topic):
    # await expensive_api_call(topic)
    return f"Data for {topic}"

async_runnable = RunnableLambda(fetch_data)
output = await async_runnable.ainvoke("AI")

# In chains: seamless composition
chain = prompt | model | RunnableLambda(lambda x: x.upper())
```

RunnableSequence: Explicit Composition

Explicitly Composed Sequences

```python
from langchain_core.runnables import RunnableSequence

# The pipe operator creates RunnableSequence
chain1 = prompt | model | parser

# Explicit construction
chain2 = RunnableSequence(
    steps=[prompt, model, parser]
)

# Both are equivalent
output1 = chain1.invoke({"topic": "AI"})
output2 = chain2.invoke({"topic": "AI"})

# Access individual steps
for i, step in enumerate(chain2.steps):
    print(f"Step {i}: {step}")

# Debug: see intermediate outputs
# (Use .stream() to see step-by-step execution)
```

RunnableBranch: Conditional Routing

If-Then-Else Logic

```python
from langchain_core.runnables import RunnableBranch

# Create conditions and routes
branches = RunnableBranch(
    (lambda x: "python" in x.lower(), python_chain),
    (lambda x: "javascript" in x.lower(), js_chain),
    default_chain  # fallback
)

# Or with RunnableBranch.route_on_type() helper
from langchain_core.messages import AIMessage, HumanMessage

branches = RunnableBranch(
    (lambda x: isinstance(x, HumanMessage), human_handler),
    (lambda x: isinstance(x, AIMessage), ai_handler),
    unknown_handler
)

# Invoke
output = branches.invoke("Show me Python code")
# Routes to python_chain
```

Branching: Complex Patterns

Multi-Level Routing

```
# Dynamically choose chain based on input
def route_model(question):
    if "quantum" in question.lower():
        return "chain_for_physics"
    elif "biology" in question.lower():
        return "chain_for_biology"
    return "default_chain"

# Combined with pipe
chain = (
    {"question": RunnablePassthrough()}
    | RunnableBranch(
        (lambda x: "quantum" in x["question"].lower(), physics_chain),
        (lambda x: "biology" in x["question"].lower(), bio_chain),
        RunnableLambda(lambda x: general_chain.invoke(x["question"]))
    )
)

output = chain.invoke("What is quantum entanglement?")
# Routes to physics_chain
```

Fallbacks: Graceful Degradation

Fallback Chains

```python
from langchain_core.runnables import RunnableLambda

# Primary chain
primary = prompt | model

# Fallback chain (e.g., cheaper model or API)
fallback = prompt | ChatOpenAI(model="gpt-3.5-turbo")

# Combine with .with_fallbacks()
chain = primary.with_fallbacks([fallback])

# If primary fails (timeout, error), try fallback
output = chain.invoke({"topic": "AI"})

# Multiple fallbacks: try in order
chain = (
    primary
    .with_fallbacks([fallback1, fallback2, fallback3])
)

# Configure exception types to catch
```

Retry Logic: Resilience

Automatic Retries

```python
# Retry with exponential backoff
chain = (
    prompt | model
).with_retry(
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=1, min=1, max=10),
    retry=retry_if_exception_type(RateLimitError)
)

# Or use tenacity directly
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(stop=stop_after_attempt(3))
def call_chain(input_data):
    return chain.invoke(input_data)

# Retry on specific exceptions
chain = (
    prompt | model
).with_retry(
    retry=retry_if_exception_type((TimeoutError, ConnectionError))
```

Configurable Fields: Runtime Flexibility

Parameterize at Runtime

```python
from langchain_core.runnables import ConfigurableField

# Make temperature configurable
model = ChatOpenAI(model="gpt-4", temperature=0.7).configurable_fields(
    temperature=ConfigurableField(
        id="temperature",
        name="Temperature",
        description="Sampling temperature",
    )
)

# Invoke with custom config
output = model.invoke(
    "Explain gravity",
    config={"configurable": {"temperature": 0.1}}
)

# More common: configurable alternatives
model_selector = ChatOpenAI().configurable_alternatives(
    ConfigurableField(id="model", name="Model"),
    gpt4=ChatOpenAI(model="gpt-4"),
```

Binding Parameters: Partial Application

.bind() for Fixed Values

```python
from langchain_openai import ChatOpenAI

# Create model
model = ChatOpenAI(model="gpt-4")

# Bind temperature (like functools.partial)
model_cold = model.bind(temperature=0.1)

output = model_cold.invoke("Summarize AI ethics")
# Runs with temperature=0.1 (not 0.7)

# Bind multiple parameters
model_strict = model.bind(
    temperature=0.0,
    max_tokens=100,
    top_p=0.9
)

# Useful in chains: bake in parameters
chain = prompt | model_strict | parser
```

Complex Chain Architecture

Multi-Step with Parallel Branches

```
# Scenario: QA system with retrieval + re-ranking + synthesis
retriever = vectorstore.as_retriever()
reranker = Reranker()  # Custom Runnable

chain = (
    {"question": RunnablePassthrough()}
    | {
        "question": RunnablePassthrough(),
        "retrieved": retriever,
        "reranked": RunnableLambda(lambda x: reranker.rank(x["retrieved
            "]))
    }
    | prompt  # Prompt templates work with dicts
    | model
    | StrOutputParser()
)

# Flow:
# 1. Input: question string
# 2. Parallel: retrieve docs, re-rank them
# 3. Feed to prompt (question + reranked docs)
```

Debugging Chains

Inspecting Intermediate Steps

```python
# Method 1: Print in RunnableLambda
chain = (
    prompt
    | RunnableLambda(lambda x: print(f"Prompt: {x}") or x)
    | model
    | RunnableLambda(lambda x: print(f"Model output: {x}") or x)
    | parser
)

# Method 2: Use .stream() for step-by-step
for chunk in chain.stream({"topic": "gravity"}):
    print(chunk)

# Method 3: Inspect schemas
print(chain.input_schema.schema())
print(chain.output_schema.schema())

# Method 4: LangChain debugging callback
from langchain_core.callbacks import StdOutCallbackHandler

output = chain.invoke(
```

Performance: Batch vs Loop

Throughput Optimization

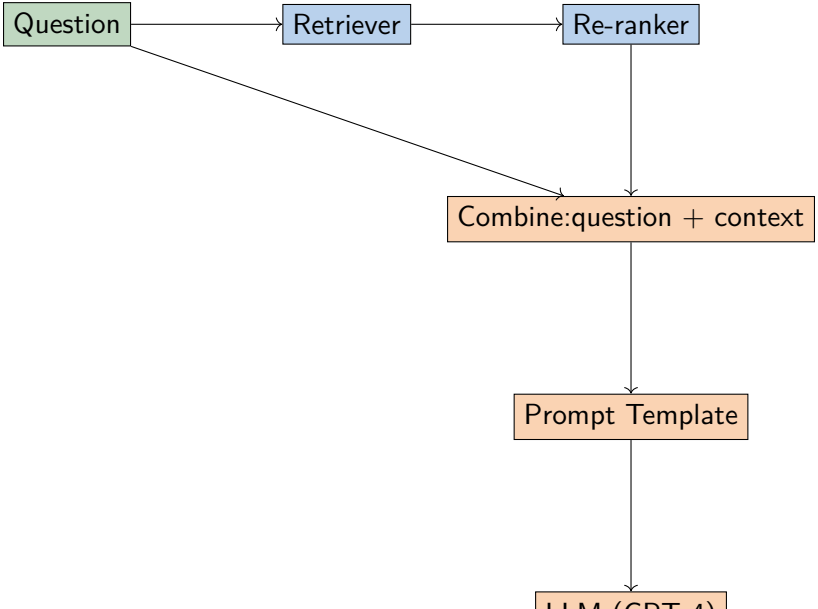
```python
import time

# Slow: sequential invoke
inputs = [{"topic": f"Topic {i}"} for i in range(100)]
start = time.time()
outputs = [chain.invoke(inp) for inp in inputs]
# ~100 LLM calls, serial: might take 3-5 minutes

# Fast: batch processing
start = time.time()
outputs = chain.batch(inputs, config={"max_concurrency": 10})
# 10 parallel requests, then next batch: ~30-60 seconds

# Even faster: batch with streaming
for outputs in chain.batch_as_completed(
    inputs,
    config={"max_concurrency": 20}
):
    # Process outputs as they complete
    print(outputs)
```

Complex Chain Data Flow (TikZ Diagram)



LCEL vs Legacy Chain Classes

Aspect	LCEL (Runnable)	Legacy Chain
Composition	`a | b`	Chain subclassing
Interface	invoke, batch, stream	run, call, apply
Async	First-class	Bolted-on
Streaming	Built-in	Inconsistent
Type safety	Schemas	Weak
Debugging	Easy	Hard
Learning curve	Low	Medium
Performance	Optimized	Baseline

Bottom line: Use LCEL for all new work. Legacy chains are deprecated.

Migration Example: Legacy to LCEL

Before (Legacy)

```python
from langchain.chains import LLMChain

chain = LLMChain(
    llm=model,
    prompt=prompt,
    output_parser=parser,
    verbose=True
)
result = chain.run(topic="AI")
```

After (LCEL)

```python
# Simpler, composable, type-safe
chain = prompt | model | parser
result = chain.invoke({"topic": "AI"})
```

Key Takeaways

1. **LCEL is the composition language** for LangChain — use for all new applications
2. **Runnable protocol** is universal — invoke, batch, stream, + async variants
3. **Pipe operator** — makes chains readable and composable
4. **Dict syntax** creates parallel branches; RunnableBranch adds conditionals
5. **RunnableLambda, Passthrough** give fine-grained control over data flow
6. **Fallbacks & retries** make chains production-ready
7. **Batch** for throughput; stream for UX; async for scalability
8. **Type safety** via schemas enables early error detection
9. **LCEL scales** from simple 1-line chains to complex multi-step systems

Philosophy: Composability over complexity. Simplicity over magic.

Quick Reference

Common Patterns

```
# Basic chain
prompt | model | parser

# Parallel (dict shorthand)
{"key": runnable1, "key2": runnable2}

# Passthrough (identity)
RunnablePassthrough()

# Lambda (wrap function)
RunnableLambda(lambda x: ...)

# Branching (conditional)
RunnableBranch((predicate, chain), default)

# Binding (partial application)
model.bind(temperature=0.1)

# Fallback (error handling)
chain.with_fallbacks([fallback1, fallback2])
```

Self-Check Questions

1. What is the Runnable protocol? Name all six core methods.
2. Why use `batch()` instead of a loop of `invoke()` calls?
3. How does the pipe operator — work? What does it create under the hood?
4. When would you use RunnablePassthrough vs RunnableLambda?
5. How do you add error recovery to a chain?
6. Name three ways to make a chain configurable at runtime.
7. What's the difference between .stream() and .batch()?
8. How do you debug an LCEL chain to see intermediate outputs?

Answer Key (Summary)

1. invoke, batch, stream, ainvoke, abatch, astream
2. `batch()` uses concurrency; much faster for multiple inputs
3. Pipe creates `RunnableSequence`; composes left-to-right
4. Passthrough = identity; Lambda = wraps Python functions
5. Use `.with_fallbacks()` and `.with_retry()`
6. `.bind()`, `.configurable_fields()`, `.configurable_alternatives()`
7. `stream()` shows tokens in real-time; `batch()` processes many in parallel
8. Inject `RunnableLambda(print)` steps, use `.stream()`, inspect schemas

Resources & Further Reading

- **Official Docs:** `https://python.langchain.com/docs/expression_language`
- **GitHub:** `https://github.com/langchain-ai/langchain`
- **Runnable API:** `https://api.python.langchain.com/en/latest/runnables/`
- **LangSmith (debugging):** `https://smith.langchain.com/`

Next steps:

- Build a simple RAG chain with LCEL
- Experiment with streaming in a web app
- Profile batch() performance vs invoke() loops
- Try RunnableBranch for multi-path routing
- Explore callbacks for observability

Version: LangChain v1.2.x (March 2026)

Questions?

Deep-dive conversation ready to continue on:

- Advanced streaming patterns
- Custom Runnable implementations
- Performance tuning for production
- Integration with tools, agents, memory